

POSTECH

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 01 -

FUNDAMENTOS DE WEB E APIS REST

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	35
O QUE VOCÊ VIU NESTA AULA?	38
REFERÊNCIAS.....	39

EMSE

O QUE VEM POR AÍ?

Imagine a internet como uma grande estrada por onde nossos dados viajam. Nesta aula, vamos voltar às origens da Web e entender por que protocolos como HTTP evoluíram ao longo do tempo, tudo isso para, no fim, aplicarmos esses conhecimentos em APIs REST voltadas a Machine Learning.

Você vai descobrir como surgiu a Web lá nos anos 90 e acompanhar a evolução do HTTP do simples ao sofisticado. Em seguida, veremos o que faz de uma API RESTful um padrão de ouro na comunicação entre sistemas e como ela se compara a outras abordagens como GraphQL, gRPC e até os mais antigos SOAP/RPC. Também vamos dissecar uma requisição HTTP e seguir passo a passo o caminho de uma chamada a um modelo de Machine Learning através de uma API.

HANDS ON

Nessa disciplina avançaremos da teoria para a prática, mostrando como projetar e consumir APIs de forma robusta.

Você verá, passo a passo, boas práticas de design e armadilhas comuns, tudo isso com exemplos do mundo real e, claro, teremos um momento hands on para pôr a mão na massa e experimentar a criação ou consumo de uma API.



SAIBA MAIS

História da Web e evolução do HTTP

Imagine que você treinou um modelo poderoso: agora, como você faz para tornar ele escalável e útil para o mundo? O desafio aqui não é apenas predição, mas integração. E você pode estar se perguntando: por que estou estudando uma tecnologia dos anos 90? Simples, todo esse universo de APIs e serviços web só existe por causa da Web em si.

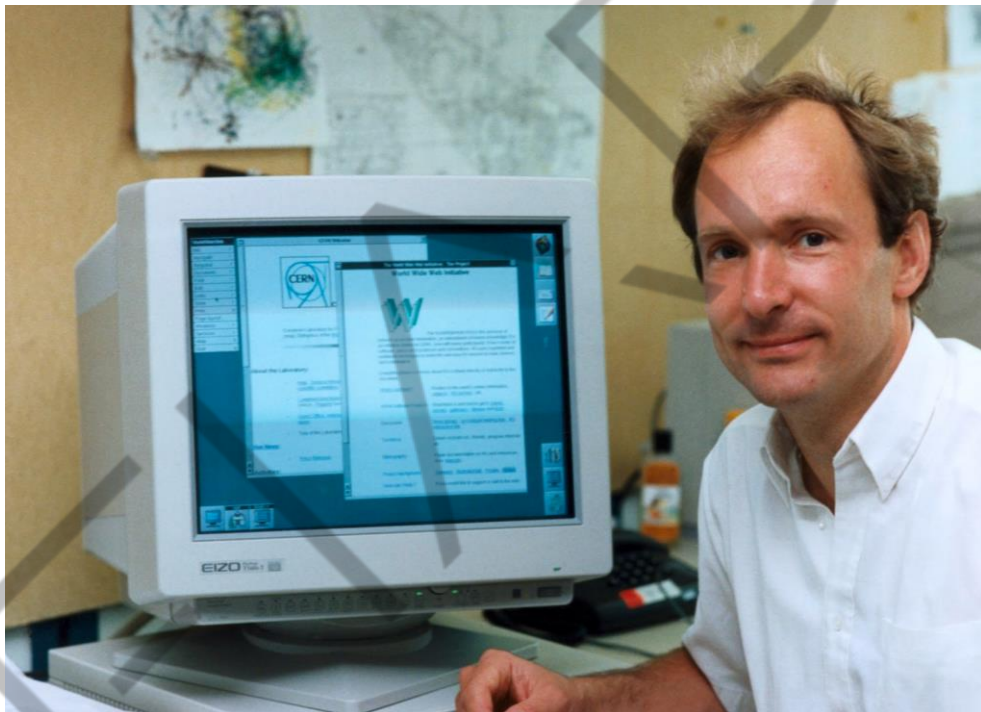


Figura 1 – Tim Berners-Lee
Fonte: Toledo (2026)

A World Wide Web surgiu no final dos anos 1980 quando Tim Berners-Lee propôs um sistema de hipertexto sobre a Internet (inicialmente chamado “Mesh”). Em 1990, ele implementou os primeiros componentes: o HTML para páginas e o protocolo HTTP para transferir conteúdo, além do primeiro browser (chamado WorldWideWeb) e servidor web (httpd).

HTTP/0.9 - Apenas GET, Sem Status

A versão original do HTTP (depois chamada HTTP/0.9) era extremamente simples: suportava apenas o método GET e retornava o conteúdo da página sem cabeçalhos ou códigos de status. Era como uma linha direta: você pedia um documento e ele vinha, ponto final, sem dizer se deu tudo certo ou não, pois não havia código 200, 404, nada disso!

HTTP/1.0 - Cabeçalhos e Códigos de Status

Com o crescimento da Web, o HTTP precisou ficar mais esperto. Em 1996 veio o HTTP/1.0, introduzindo cabeçalhos, códigos de status (finalmente um jeito de dizer "200 OK, deu certo!" ou "404 Not Found, não achou") e a ideia de versionar cada requisição.

HTTP/1.1 - Conexões persistentes, Pipeline, Cache

Isso permitiu respostas mais informativas e melhor uso de cache. Logo em 1997, chegou o HTTP/1.1, que virou o padrão dominante por décadas. O HTTP/1.1 trouxe conexões persistentes (keep-alive), permitindo reutilizar a mesma conexão TCP para múltiplas requisições. Pense em não precisar abrir uma nova “porta” para cada pedido, acelerando muito a comunicação.

Ele também suportou requisições em pipeline (enviar várias de uma vez) e melhorou controles de cache. Foi o HTTP/1.1 que tornou a web escalável de verdade, lidando bem com múltiplos usuários simultâneos e infraestrutura como proxies e caches ao longo do caminho.

HTTP/2 - Binário, Multiplexação, Compressão de Cabeçalhos

Nas décadas seguintes, a busca por desempenho continuou puxando novas versões. O HTTP/2 (padronizado em 2015) trouxe mudanças maiores: em vez de texto puro, passou a usar um formato binário e introduziu multiplexação, ou seja, vários

pedidos podem trafegar em paralelo na mesma conexão sem um bloquear o outro (resolvendo o problema do head-of-line blocking do HTTP/1.x).

Além disso, o HTTP/2 implementou compressão de cabeçalhos para reduzir tamanho e um recurso chamado server push, em que o servidor pode mandar respostas não solicitadas para o cliente se antecipando (útil em alguns casos para desempenho).

HTTP/3 - QUIC/UDP, Baixa Latência, Sem Bloqueios

E mais recentemente, o HTTP/3 (oficializado em 2022) mudou radicalmente o transporte: em vez de TCP, usa o protocolo QUIC sobre UDP. Isso elimina algumas limitações do TCP (como aqueles handshakes demorados) e lida melhor com conexões instáveis; o HTTP/3 consegue manter a performance mesmo com perdas de pacotes e suporta multiplexação sem bloqueios a nível de transporte.

Por que essa evolução toda importa para a gente, que quer expor modelos de ML via APIs? Porque cada melhoria no HTTP abre possibilidades para APIs mais eficientes e robustas. Conexões persistentes e multiplexadas (HTTP/2 e 3) significam que você consegue atender várias requisições de previsão em paralelo sem tanta sobrecarga de abrir conexões, ótimo para serviços de ML de alta demanda.

Compressão ajuda a enviar respostas (ou requisições) mais rápido, o que é útil se seu modelo cospe um JSON grandinho. E o menor tempo de latência do HTTP/3 pode fazer diferença em uma aplicação de tempo real, por exemplo. Em suma, conhecer a história e as versões do HTTP – de um protocolo stateless e simples, até um sistema binário criptografado e otimizado – nos ajuda a projetar melhor nossas APIs, aproveitando o que cada versão oferece. Afinal, um(a) engenheiro(a) de ML que sabe esses “truques” pode configurar seu serviço para tirar proveito do HTTP/2, por exemplo, e ganhar performance.

O que são APIs RESTful: Definição e Princípios

Falamos de HTTP, mas e as APIs REST nisso tudo? Bom, REST (Representational State Transfer) não é uma tecnologia em si, mas um estilo arquitetural para comunicação web definido por Roy Fielding em sua famosa dissertação de 2000. Ele propôs um conjunto de restrições de design para construir sistemas distribuídos escaláveis na web. Quando criamos uma API RESTful, significa que estamos seguindo esses princípios em cima do HTTP.



Figure 5-9. REST Derivation by Style Constraints

Roy T Fielding

REST
2000

Figura 2 – Roy Fielding - REST
Fonte: Koriyama (2019)

Um serviço REST expõe recursos que podem ser identificados por URLs (URIs, para ser mais preciso) e permite manipular esses recursos usando operações uniformes, mais precisamente os verbos HTTP padrão como GET, POST, PUT, DELETE etc.

Em vez de cada API inventar seu jeito de fazer as coisas, como acontecia no passado com alguns RPCs ou SOAP com mil operações complexas, o REST abraça o que já existe no HTTP de forma natural.

Por exemplo: se temos um recurso “clientes”, esperamos que `GET /clientes` traga a lista de clientes, `POST /clientes` crie um novo cliente, `GET /clientes/123` pegue o cliente com ID 123, `PUT /clientes/123` atualize e `DELETE /clientes/123` remova. Notou a consistência? O URL identifica o que você quer, o verbo HTTP diz o que fazer. Essa uniformidade faz com que qualquer um que saiba HTTP consiga usar a API sem um manual de 500 páginas.

Os princípios do REST incluem também algumas ideias-chave:

Cliente-Servidor e Stateless

O servidor não guarda estado da sessão do cliente entre requisições, cada pedido é independente. Cada requisição HTTP deve conter todas as informações necessárias (por exemplo, dados de autenticação, parâmetros etc.) sem depender de contexto de sessão armazenado no servidor.

Recursos e Identificação

Tudo é tratado como um recurso (entidades, objetos ou dados) acessível por um identificador (um URI). Boas práticas sugerem estruturar URIs com substantivos, representando coleções e itens (ex: `/clientes/123/pedidos`) em vez de verbos ou ações. Cada recurso pode ter múltiplas representações, por exemplo, JSON ou XML, negociadas via content-type.

Verbos HTTP padronizados

Utiliza os métodos HTTP de forma consistente para as operações: GET (leitura), POST (criação), PUT (substituição), PATCH (atualização parcial) e DELETE (remoção) de recursos, entre outros. A semântica correta dos verbos é importante – por exemplo, não usar GET para modificar estado, nem usar POST para obter dados, mantendo assim a comunicação intuitiva e compatível com caches, navegadores etc.

Cacheável, Idempotência e Segurança

Em REST, certos métodos devem ser idempotentes, significando que múltiplas execuções produzem o mesmo efeito que uma única (excluindo efeitos colaterais como logs). Pelo padrão HTTP, GET, PUT, DELETE e PATCH são idempotentes (com a mesma entrada, repetidas vezes retornam o mesmo resultado, salvo talvez um 404 em uma segunda deleção). Além disso, GET (e HEAD) são seguros, não devendo modificar estado do servidor. Sempre que possível devemos aproveitar caches HTTP para respostas repetidas, por exemplo. Seguir essas regras permite reentrâncias e replays sem efeitos adversos.

Representações e Formatos de Dados

Um recurso pode ter diferentes representações. APIs RESTful frequentemente usam JSON como formato de dado pelo seu peso leve e compatibilidade natural com JavaScript, mas não estão limitadas a ele – XML, CSV ou formatos binários podem ser usados conforme a necessidade. O cliente e o servidor negociam o formato via cabeçalhos HTTP (Accept, Content-Type). Por exemplo, um mesmo endpoint pode aceitar e responder tanto em JSON quanto XML, dependendo do header fornecido pelo cliente.

Hipermídia (HATEOAS): Hypermedia as The Engine of Application State

Existe também o conceito de codificação sob demanda e hypermedia (HATEOAS), uma restrição onde as respostas incluem links hipertexto para outras operações relacionadas, permitindo que o cliente descubra dinamicamente como interagir com mais recursos através dos links fornecidos, em vez de depender de documentação externa.

Em resumo, uma API é considerada RESTful quando adere a esses princípios arquiteturais: separação clara entre cliente e servidor, ausência de estado de sessão, interface uniforme (mesmos verbos para todos os recursos), recursos bem definidos

por URIs, operações idempotentes quando devido, respostas auto-descritivas (via códigos HTTP) e, idealmente, hipermídia para navegação.

Uma API REST bem feita se parece com a Web em miniatura. Ela faz o cliente navegar por recursos através de links e operações padrão.

REST vs. outras abordagens (GraphQL, gRPC, SOAP, RPC)

Tudo bem, REST é o rei da popularidade, mas ele não é a única opção para se construir APIs: existem outras arquiteturas de APIs que se adequam melhor a certos cenários, especialmente em aplicações de Machine Learning e Microsserviços.

A seguir, comparamos brevemente REST com GraphQL, gRPC, SOAP e a ideia geral de RPC, destacando vantagens, limitações e usos ideais em contexto de ML.

GraphQL: é uma linguagem de consulta para APIs criada pelo Facebook em 2015 e depois aberta para a comunidade. Em vez de ter vários endpoints, cada um retornando um conjunto fixo de dados, o GraphQL expõe um único endpoint (geralmente `/graphql`) em que o cliente faz queries especificando exatamente os dados que quer e o servidor retorna só aquilo, no formato JSON.



Figura 3 – GraphQL
Fonte: Google Imagens (2026)

Pense no GraphQL como um “self-service por encomenda”: o cliente monta o “prato” de dados do jeito que precisa. A grande vantagem é permitir fetch de dados altamente customizado e evitar overfetching (receber dados demais que não vai usar) e underfetching (ter que fazer várias chamadas diferentes para juntar todos os dados que precisa). Com GraphQL você pode aninhar consultas e, por exemplo, pegar um usuário e junto os últimos pedidos dele, tudo em uma tacada só e somente com os campos solicitados. Além disso, o GraphQL trabalha com um esquema fortemente tipado e introspectivo, o que ajuda a validar as consultas e permite gerar documentação e clients automáticos facilmente.

No contexto de ML, o GraphQL brilha se você tem muitos tipos de dados relacionados (metadados de modelos, resultados diferentes, informações de experimentos...) e clientes distintos precisando de subconjuntos diferentes desses dados. Por exemplo, imagine um sistema de monitoramento de experimentos de ML: um dashboard pode querer alguns campos dos resultados, outro app pode querer outros – com GraphQL, cada um pede exatamente o que quer, evitando transferência de dados irrelevantes.

Mas nem tudo são flores e essa flexibilidade do GraphQL traz desafios de cache: requisições GraphQL tendem a ser únicas (porque cada cliente pede campos diferentes), então caches HTTP tradicionais (que funcionam bem com GETs previsíveis) ficam menos efetivos. Isso também adiciona complexidade no servidor: você precisa definir schemas, resolver funções (resolvers) para cada campo etc. Pode ser overkill se o seu serviço é simples, como um modelo de previsão direto.

Em muitos casos de ML, o GraphQL não é comum para comunicação entre serviços no backend; ele é mais usado entre cliente front-end e servidor, especialmente em apps com interface rica onde você quer otimizar o número de requisições, como por exemplo uma single-page app que quer minimizar idas e vindas. Se o seu caso é oferecer um serviço preditivo simples (tipo “dado X devolve Y”), REST normalmente basta e o GraphQL seria complexo demais para algo tão direto.

Em resumo: o GraphQL é ótimo para APIs de dados complexos e flexibilidade de consulta, mas talvez seja canhão demais para APIs de predição simples.

gRPC: é um framework de RPC de alta performance desenvolvido pelo Google, lançado em 2016. Ele usa Protocol Buffers (um formato binário super eficiente) e por baixo dos panos roda sobre HTTP/2. De cara, a vantagem do gRPC é velocidade e eficiência: as mensagens são binárias e compactas, a conexão é persistente e multiplexada (HTTP/2) e ele suporta chamadas do tipo streaming (enviar e receber dados de forma contínua) de forma nativa.



Figura 4 – gRPC
Fonte: Google Imagens (2026)

O gRPC trabalha a partir de um contrato (.proto) do qual você gera código cliente e servidor automaticamente. Isso garante que ambos os lados entendam perfeitamente a mensagem (tipagem forte) e facilita implementar, sem erros de “ah, esqueci esse campo”.

Para sistemas de ML, o gRPC é uma maravilha dentro de infraestruturas de microsserviços: pense em dezenas de serviços conversando entre si com latência baixíssima, chamando inferências em tempo real ou mandando fluxo de dados para um modelo online. Por exemplo, se você tem um pipeline de processamento de dados de sensores IoT alimentando um modelo de detecção de anomalias, o gRPC seria uma ótima escolha pela eficiência.

Grandes plataformas de ML internas, como TensorFlow Serving e Seldon Core, inclusive suportam gRPC além de REST, exatamente para casos em que desempenho é crítico.

As limitações: por usar HTTP/2 e comunicação binária, o gRPC não é consumível diretamente por navegadores ou clientes simples – você não faz um gRPC com um fetch do JavaScript, por exemplo, sem um gateway especial.

Geralmente é usado entre servidores/backends ou com stubs gerados (aquelas classes que o proto gera) nos clientes. Depurar pode ser mais difícil, já que a mensagem não é legível facilmente (ao contrário de um JSON que você consegue inspecionar). Além disso, o gRPC exige TLS obrigatório, o que é bom para segurança, mas dá um passo a mais de configuração (certificados etc.) mesmo em ambientes internos.

Resumindo: o gRPC é indicado quando desempenho e contrato rígido importam muito. Se você tem um serviço de ML com altíssima QPS (queries per second) servindo outros serviços internos, o gRPC pode ser a melhor escolha. Já para expor APIs para consumo público ou de clientes diversos, o REST/JSON costuma ser preferível (mais compatível e simples de integrar).

Muitas vezes a solução híbrida é usada: por exemplo, um serviço de modelo disponibiliza gRPC internamente para outros serviços chamarem rápido, mas expõe um gateway REST na frente para atender aplicações web/mobile que esperam JSON.

SOAP: aqui entramos no túnel do tempo. O SOAP (Simple Object Access Protocol) foi muito popular nos anos 2000 como padrão de integração entre serviços. Ele é baseado em XML e segue um protocolo bem rígido. Uma chamada SOAP envolve enviar um XML dentro de um envelope, definindo a operação e os parâmetros, e a resposta vem também em XML.



Figura 5 – Soap
Fonte: Google Imagens (2026)

O SOAP tipicamente usa HTTP como transporte (embora possa usar outros, como SMTP), mas ele ignora grande parte do “significado” do HTTP. Ele usa quase tudo via POST e deixa a semântica dentro do XML.

Vantagens do SOAP: ele tem contratos formais via WSDL, esquemas XML para validar tudo e um ecossistema gigante de ferramentas empresariais (muitos sistemas legados de banco, governo, usaram e usam SOAP). O SOAP também especifica muitas extensões para segurança (WS-Security), transações distribuídas etc., que foram úteis em ambientes corporativos complexos.

Limitações: verbosidade. XML por si só já é mais pesado que JSON e o envelope SOAP adiciona camadas extras, então as mensagens ficam enormes. Implementar e manter um serviço SOAP é complexo – geralmente precisa de frameworks pesados (como Apache Axis, .NET WCF etc.).

Hoje, o SOAP é considerado pesado e meio ultrapassado para novos projetos web. No contexto de ML, dificilmente você vai optar por SOAP a não ser que precise integrar com algum sistema legado que já use SOAP. Por exemplo, se um banco possui um sistema de crédito antigo exposto via SOAP, você pode acabar tendo que consumir ou expor algo compatível para encaixar um modelo preditivo naquele fluxo, mas isso é exceção.

Em geral, para novos serviços de predição ou data products, REST/JSON ou gRPC são preferidos por serem bem mais leves e simples.

RPC simples (JSON-RPC, XML-RPC ou customizados): RPC, chamada de procedimento remoto, é a ideia base tanto do SOAP quanto do gRPC, que é você chamar uma função em outro servidor como se estivesse local. Além dos grandes frameworks, existem implementações simples de RPC, como o JSON-RPC (que envia requisições JSON indicando método e parâmetros) ou XML-RPC (um precursor do SOAP). E claro, alguém pode criar seu próprio estilo de RPC customizado usando HTTP: por exemplo, fazer um único endpoint /api que aceita POST e no corpo JSON você manda { "acao": "deletar_usuario", "id": 5 }.

Quais as vantagens? Pode ser extremamente simples de implementar em uma necessidade específica – às vezes em um protótipo interno, você só quer algo rápido: um serviço de ML expõe um método predict e pronto, sem se preocupar em estruturar recursos e verbos. O overhead pode ser mínimo e funcionar bem em um ambiente controlado. Porém, a falta de padrão é o calcanhar de Aquiles: cada RPC customizado pode ter convenções totalmente diferentes.

Você perde muitos benefícios do HTTP – caches, ferramentas prontas, semântica uniforme – porque do ponto de vista do protocolo todas as chamadas são, digamos, POST para o mesmo URL então, do lado de fora parece tudo igual, quem consome tem que conhecer exatamente o contrato específico. Isso aumenta o acoplamento: se o servidor muda a forma do método ou parâmetro, quebra o cliente, e não havia padrão claro pra começar. Em ML, um RPC custom até poderia surgir naquele cenário de “microsserviços altamente performáticos” onde querem evitar qualquer overhead, mas honestamente, o gRPC já cobre essa demanda entregando performance com contrato forte, então poucos reinventariam a roda hoje.

Assim, JSON-RPC e similares são pouco usados externamente; REST ou GraphQL cobrem a maioria dos casos de API web e gRPC cobre os cenários de alto desempenho.

Quando usar cada um? Depende, claro, dos requisitos

Use REST para expor modelos de ML ao mundo web de forma padrão. É fácil integrar com frontends, amplamente compatível (praticamente toda linguagem “fala” HTTP/JSON) e ótimo para serviços do tipo request-response tradicionais de predição. Ex.: um serviço de classificação de imagens acionado via POST /predicao com uma imagem e que retorna JSON com o resultado – REST dá conta do recado de forma simples e entendível.

Considere GraphQL se você realmente precisa que diferentes clientes customizem muito os dados retornados ou quando há muitos recursos relacionados. Por exemplo, imagine um serviço que agrega resultados de vários modelos e quer devolver tudo junto: em vez de várias chamadas REST diferentes, um GraphQL poderia permitir ao cliente pegar tudo o que precisa em uma requisição bem sob medida. Porém, para um endpoint simples de previsão, o GraphQL tende a ser exagero.

Use gRPC para comunicação entre serviços de ML em uma arquitetura de microsserviços, em que eficiência e baixa latência são críticas. Ex.: um pipeline interno que processa dados de sensores e alimenta modelos – o gRPC vai garantir throughput e tempo de resposta ótimos. Plataformas de serving como Seldon e TensorFlow Serving oferecem gRPC exatamente por isso. Mas lembre-se: se for expor um serviço para consumo direto de apps web ou de terceiros, talvez ofereça um REST por cima, a não ser que seu público consiga lidar com gRPC. Muitas empresas têm gRPC internamente e REST externamente como fachada.

O SOAP é recomendado apenas se for obrigatório (legados, integração com quem exige). Caso contrário, os caminhos modernos já suprem quase tudo que o SOAP fazia, com menos complexidade. RPC custom: evite a menos que seja algo interno, temporário ou muito bem justificado. Se precisar de simplicidade, às vezes até um REST sem ser perfeito já resolve, ou mesmo um gRPC leve.

Criar um protocolo do zero raramente vale a pena, você acaba reimplementando coisas que os outros já solucionaram.

Anatomia de uma requisição HTTP e chamada de modelo de ML via API

Vamos agora “abrir” uma requisição HTTP para ver suas partes, porque isso é fundamental para entender como enviamos dados para um modelo e recebemos resposta. Uma mensagem HTTP tem uma estrutura bem definida: começa com uma linha inicial, seguida de cabeçalhos, e depois um corpo (opcional). No caso de uma requisição HTTP, a primeira linha diz o verbo, o caminho (URL) e a versão do protocolo. Por exemplo:

```
POST /api/predicao HTTP/1.1
```

Isso indica: método POST, no caminho /api/predicao , usando HTTP 1.1. Depois dessa linha, vêm os cabeçalhos, cada um em uma linha no formato Campo: valor.

Alguns cabeçalhos comuns em requisições são, por exemplo:

```
Host: meu-servidor.com
```

```
Content-Type: application/json
```

```
Authorization: Bearer <token>
```

Os cabeçalhos fornecem metadados sobre a requisição – desde qual host você está chamando, o tipo de conteúdo do corpo (se vai mandar JSON, XML etc.), informações de autenticação, cache, idioma, entre outros. Após listar todos os cabeçalhos, uma linha em branco os separa do corpo da mensagem. Se o método suporta/envolve envio de dados (como POST, PUT, PATCH), aí em seguida vem o corpo da requisição, que pode ser um texto (JSON, XML) ou binário (por exemplo, se você estiver enviando uma imagem ou arquivo). Um exemplo de corpo JSON simples poderia ser:

```
{ "entrada": [1.2, 5.4, 3.3] }
```

Esse seria o payload que estamos mandando para o servidor.

E o que o servidor faz? Ele processa o pedido e devolve uma resposta HTTP. A resposta também tem uma estrutura: começa com uma linha de status (que contém o código de status e uma frase descritiva), depois cabeçalhos de resposta e

opcionalmente um corpo. Por exemplo, uma resposta típica de sucesso pode começar com:

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

E depois vir um corpo, digamos:

```
{ "resultado": 42 }
```

Aqui o 200 OK indica sucesso, o content-type diz que estamos retornando JSON e o corpo traz o dado pedido (neste exemplo imaginário, 42 como resultado).

Essa estrutura de requisição/resposta é a base para chamar modelos de Machine Learning via API. Expor um modelo de ML como um serviço REST significa que o cliente (pode ser seu front-end, um aplicativo móvel ou outro sistema) envia uma requisição HTTP contendo os dados de entrada do modelo e o servidor retorna a inferência do modelo dentro de uma resposta HTTP padronizada.

Vamos tornar isso concreto com um exemplo. Suponha um modelo de ML que prediz probabilidade de diabetes de um paciente a partir de exames. Uma chamada REST para este modelo poderia ser:

```
POST /api/v1/predict HTTP/1.1
```

```
Host: ml.exemplo.com
```

```
Content-Type: application/json
```

```
{
```

```
  "pregnancies": 2,
```

```
  "glucose": 148,
```

```
  "blood_pressure": 72,
```

```
  "skin_thickness": 35,
```

```
  "insulin": 0,
```

```
  "bmi": 33.6,
```

```
  "diabetes_pedigree_function": 0.627,
```

```
"age": 50  
}
```

O que temos aqui? O cliente faz um POST no endpoint `/api/v1/predict` do host `ml.exemplo.com`, indicando que está enviando JSON. No corpo, está passando os features do paciente (gravidez, glicose, pressão etc.). Esse é exatamente o input que o modelo de ML precisa para gerar uma predição.

No servidor, provavelmente temos uma aplicação (em Python Flask/FastAPI, Java Spring Boot, Node etc.) que recebe essa requisição. Ela vai desserializar o JSON (transformar aquele texto em um objeto ou dicionário), talvez fazer algum pré-processamento (normalizar valores, verificar campos faltantes) e então passar os dados para o modelo preditivo. Pode ser que o modelo já esteja carregado em memória para rapidez.

Após obter o resultado da previsão, o servidor prepara a resposta HTTP. No caso de sucesso, digamos que o modelo previu que `outcome = 1` (positivo para diabetes). Poderíamos retornar:

```
HTTP/1.1 200 OK  
  
Content-Type: application/json  
  
{ "outcome": 1 }
```

Nesse caso, o cliente receberia um JSON simples indicando o resultado predito. Se `outcome: 1` representa positivo (diabetes) e 0 seria negativo, pronto, a API cumpriu seu papel: encapsulou o modelo de ML atrás de uma interface web simples.

Em um teste real, um desenvolvedor ou desenvolvedora poderia verificar essa API via comando `curl`, assim:

```
curl-X POST https://ml.exemplo.com/api/v1/predict  
  
\-H "Content-Type: application/json"  
  
\-d '{  
  
  "pregnancies": 2, "glucose": 148, "blood_pressure": 72,  
  
  "skin_thickness": 35, "insulin": 0, "bmi": 33.6,
```

```
"diabetes_pedigree_function": 0.627, "age": 50
```

```
}]'
```

Esse comando envia exatamente a requisição que descrevemos e deve receber como resposta um JSON com o resultado predito, por exemplo {"outcome": 1}.

Internamente, integrar um modelo de ML a uma API envolve essas etapas: carregar o modelo (treinado previamente) no servidor, receber os dados do request, possivelmente pré-processar (normalizar valores, converter categorias em números etc.), chamar o método de inferência do modelo e depois formatar o resultado em uma resposta HTTP.

Tudo isso precisa acontecer rápido – dentro do tempo de um request típico web (milissegundos a poucos segundos, dependendo do modelo). Para garantir desempenho, implementações comuns mantêm o modelo carregado em memória (evitando recarregar a cada request) e podem até enfileirar requisições se o processamento for pesado, usando por exemplo um worker pool de threads ou processos.

Boas práticas de design de APIs REST

Projetar uma API REST robusta e amigável é essencial, especialmente para sistemas de ML em produção onde vários clientes (e desenvolvedores(as)) vão consumir aquele serviço. Vamos passar por algumas boas práticas chave de design, com foco no contexto de expor serviços de Machine Learning.

Seguir essas dicas pode fazer a diferença entre uma API tranquila de usar/manter e um pesadelo de suporte.

Versionamento de API

Mudanças vão acontecer. Modelos de ML evoluem, novos parâmetros podem ser adicionados, formatos de input/output podem mudar conforme refinamos o serviço. Para não quebrar clientes existentes a cada atualização, é importante ter um esquema

de versão na API. O jeito mais simples e comum é versionar na URL: por exemplo incluir /v1/ no caminho base (como usamos em /api/v1/predict no exemplo). Assim, se daqui a seis meses você treinar um modelo novo com um formato de entrada diferente, pode lançar um /api/v2/predict e manter o /v1 funcionando para quem ainda não migrou.

Alternativas incluem versionar via parâmetros de query ou cabeçalhos customizados, mas o fundamental é planejar isso cedo. Documente as versões e mudanças e pratique depreciação gradual, ou seja, avise os consumidores com antecedência que algo vai mudar. Dá até para usar um cabeçalho Warning ou um campo no JSON de resposta avisando “Versão v1 será descontinuada em tal data, use v2”.

Em resumo: não pegue seus usuários de surpresa com mudanças quebrando a integração.

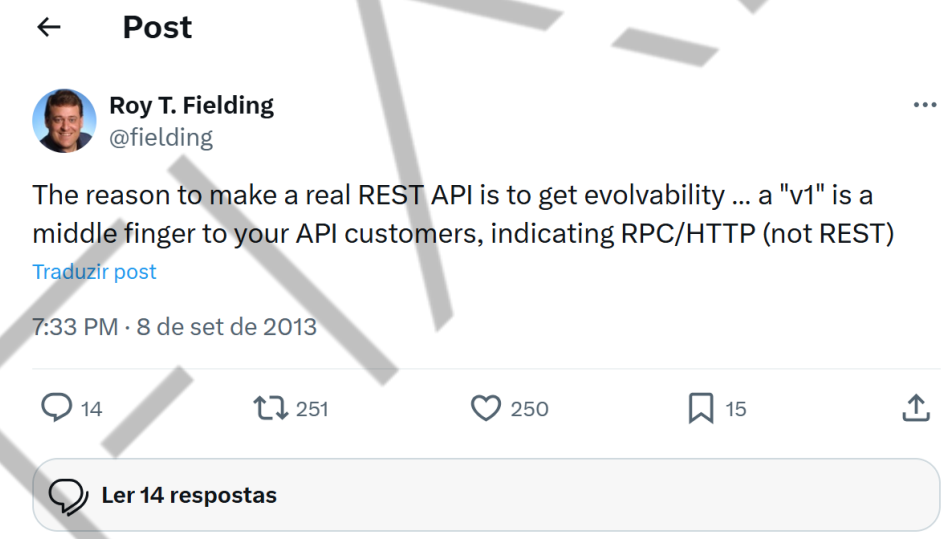


Figura 6 – PS. REST “puro” vs REST “de mercado”
Fonte: Fielding (2013)

Roy Fielding, criador do REST, defende que o verdadeiro objetivo do REST é evoluir sem quebrar contratos. Para ele, colocar /v1 na URL é um sinal de que a API está sendo tratada como um RPC (chamada remota de procedimento), não como um recurso evolutivo da Web.

Contudo, o mercado adotou uma versão mais pragmática do REST, em que /v1 é visto como uma forma simples de manter estabilidade e transparência — especialmente em sistemas com múltiplos clientes e versões coexistindo.

Design de URIs (URLs)

Uma boa API tem URLs claras e sem floreios desnecessários. Prefira substantivos no plural para recursos (ex. /clientes/123 ao invés de verbos ou frases como /getClient?id=123).

Evite enfiar ações na URL, por exemplo, ao invés de /deletarCliente/123, use o verbo HTTP DELETE em /clientes/123. Isso segue a filosofia REST de que a ação vem no método HTTP, não no caminho. Consistência é a palavra: escolha um padrão e siga. Tudo minúsculo, sem espaços, use hífen ou underscore se precisar separar, mas não misture. Não inclua extensões como .json na URL, negocie formato via cabeçalhos se precisar (geralmente nem precisa, o JSON é padrão hoje).

Lembre-se que a URL deve representar o recurso, não a operação. Então nada de /listaTodosUsuarios se você pode ter /usuarios com GET para listar. Também evite fazer URLs profundamente hierárquicas sem necessidade. Por exemplo, /loja/setor/produto/categoria/id às vezes pode ser simplificado dependendo do modelo de dados.

Cada nível hierárquico deve fazer sentido (ex. /lojas/10/setores/5/produtos se realmente produto está dentro de setor que está dentro de loja). URLs muito “verbosas” ou inconsistentes confundem quem usa a API. Em suma: mantenha-as limpas e significativas.

O uso adequado dos verbos HTTP é essencial: GET deve apenas obter dados sem alterar o servidor, POST cria ou aciona operações não idempotentes, PUT substitui recursos, PATCH atualiza parcialmente e DELETE remove. Anti-padrões comuns incluem usar GET para deletar ou POST para tudo, o que quebra a semântica do protocolo e dificulta a integração. Se uma operação não se encaixa bem nos verbos, é melhor repensar o design e representá-la como recurso, mantendo a API clara e previsível.

Os códigos de status também devem ser usados corretamente para comunicar o resultado da requisição. 200 indica sucesso, 201 criação de recurso, 204 operações sem corpo de resposta, enquanto erros do cliente devem retornar 4xx adequados (400, 401, 403, 404, 422) e falhas do servidor 500.

Além disso, é importante fornecer mensagens de erro consistentes em formato estruturado, como o padrão RFC 7807, para facilitar depuração e integração. Da mesma forma, implementar filtros, paginação e seleção de campos evita sobrecarga de dados e melhora a performance, dando flexibilidade ao consumidor.

CÓDIGO	NOME	USO/DESCRIÇÃO
200	OK	Sucesso comum (GET ou operações que não criam nada novo).
201	Created	Novo recurso criado; ideal incluir header Location com a URL do recurso.
204	No Content	Operação bem-sucedida sem resposta no corpo (ex.: DELETE ou PUT sem retorno).
400	Bad Request	Requisição malformada ou inválida (campos faltando, tipos errados).
401	Unauthorized	Falta autenticação ou token inválido (não autenticado).

403	Forbidden	Cliente autenticado, mas sem permissão/autorização para o recurso/ação.
404	Not Found	Recurso solicitado não existe (ex.: ID não encontrado).
409	Conflict	Conflito de estado (duplicação ou violação de condição atual).
422	Unprocessable Entity	Erros de validação de conteúdo (dados corretos sintaticamente, mas inválidos semanticamente).
500	Internal Server Error	Erro inesperado no servidor (exceções não tratadas, falha interna).

Tabela 1 - Tabela de Códigos de Status HTTP adequados
Fonte: Elaborado pelo autor (2026)

Autenticação e Autorização

Segurança é fundamental. Não exponha APIs sensíveis sem proteção adequada, mesmo que “ah, é só interno”. Muitas brechas acontecem em APIs internas sem vigilância. Usar autenticação tokens de API, OAuth 2.0 ou JWT são comuns hoje. Para serviços stateless, tokens JWT (JSON Web Tokens) são ótimos: o cliente faz login em algum lugar, recebe um token e passa esse token em cada requisição (no cabeçalho Authorization: Bearer token_aqui). O servidor, a cada request, valida o token (assinatura, data de expiração) e extrai informações de identificação/permissão dali. Assim você não guarda sessão no servidor (compatível com REST stateless) e ainda sim sabe quem é o usuário e seus privilégios.

Use sempre HTTPS, obviamente, para que tokens e senhas não trafeguem em texto plano. Implemente autorização granular: nem todo mundo autenticado pode acessar tudo. Você pode incluir roles/permissões dentro do token ou em outro sistema e checar no endpoint – ex.: apenas administradores podem chamar `/retrainModel` ou apagar recursos, um usuário comum só pode ver as previsões dele etc.

Retorne aos códigos adequados quando barrar: 401 se não apresentou credenciais ou inválidas, ou 403 se apresentou mas não tem autorização para aquilo.

Cacheabilidade, documentação e monitoramento

Recursos auxiliares podem ser cacheados com cabeçalhos apropriados, reduzindo latência. Documentação clara, preferencialmente com Swagger/OpenAPI, é indispensável para orientar uso correto e acompanhar versões. Por fim, monitoramento e logging com métricas e correlation IDs ajudam a diagnosticar problemas e manter governança em MLOps.

Resumindo as boas práticas: consistência e previsibilidade. Uma API é como um contrato com quem consome, se você segue padrões desde o início, todo mundo sai ganhando. E lembre-se que no contexto de ML Engineering, às vezes quem consome pode ser outro time, ou até você mesmo em uma aplicação diferente, por isso aplicar princípios sólidos de engenharia de software aqui evita dores de cabeça na integração dos modelos no mundo real.

Ferramentas para teste e consumo de APIs (Postman, curl, HTTPie)

Durante o desenvolvimento e consumo de APIs REST, especialmente quando estamos validando endpoints de modelos de ML, é essencial usar ferramentas que facilitem fazer requisições e depurar as respostas. Vamos falar de três bem populares e úteis: Postman, curl e HTTPie. Cada uma tem seu estilo, do visual à linha de comando, mas todas ajudam a brincar com sua API antes mesmo de escrever código cliente.



Figura 7 – Soap
Fonte: Google Imagens (2026)

Talvez a mais famosa ferramenta gráfica para APIs. O Postman fornece uma interface amigável onde você monta requisições HTTP, envia e inspeciona o resultado. Com ele você pode facilmente escolher o método (GET/POST/etc.), colocar a URL, adicionar cabeçalhos, corpo da requisição em diversos formatos e ver o retorno estruturado (com destaque de sintaxe, tempo de resposta, código status, tudo bonitinho). Quais as vantagens? É ótimo para exploração e documentação. Você pode salvar requisições em coleções (por exemplo, uma coleção chamada "API de Predição" com vários calls – um para /predict , outro para /health etc.) e pode até escrever testes automatizados no Postman (por meio de scripts) para verificar se a resposta contém certos valores, por exemplo.

É muito útil para criar um conjunto de exemplos e depois gerar documentação a partir disso (o Postman tem recurso de publicar docs). Também possui recursos como mock servers (para você simular a API antes mesmo dela estar pronta), gerenciamento de variáveis de ambiente (ótimo para alternar entre dev, teste, produção apenas mudando o host base, por exemplo) e até gerar código em várias linguagens para aquela requisição (facilitando integrar no seu frontend/backend depois).

Um exemplo no nosso contexto: você desenvolveu a API de predição de diabetes. No Postman, você pode criar a requisição POST com o JSON de entrada,

mandar e ver o JSON de resposta e o tempo. Se algo der erro, você ajusta e manda de novo rapidamente. O Postman permite inclusive que, se sua API requer autenticação, você configure isso facilmente – tem suporte integrado para tokens Bearer, OAuth 2.0 flows etc., sem precisar escrever todo header na mão toda hora. A principal desvantagem do Postman é ser uma aplicação relativamente pesada e às vezes para coisas muito simples ele é um exagero (abrir uma UI toda só pra fazer um GET, sabe?).

Além disso, embora tenha plano gratuito robusto, é uma ferramenta proprietária e empresas maiores às vezes preferem alternativas open source ou self-hosted por políticas, mas em geral o Postman é amplamente usado tanto por devs quanto QAs. Vamos ver ele na prática nas próximas aulas.

```
curl http://localhost:8000/
>> C:\B3-API>

StatusCode      : 200
StatusDescription : OK
Content         : {"message":"OlÃ¡, eu sou sua primeira API!"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 44
                  Content-Type: application/json
                  Date: Sun, 30 Nov 2025 14:28:46 GMT
                  Server: uvicorn

                  {"message":"OlÃ¡, eu sou sua primeira API!"}
Forms           : {}
Headers         : {[Content-Length, 44], [Content-Type, application/json], [Date, Sun,
                  30 Nov 2025 14:28:46 GMT], [Server, uvicorn]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : System.__ComObject
RawContentLength : 44

PS C:\B3-API> |
```

Figura 8 – Curl
Fonte: Elaborado pelo autor (2026)

Esse é o canivete suíço dos desenvolvedores e desenvolvedoras. O curl é um utilitário de linha de comando presente na maioria dos sistemas (Linux, Mac, até Windows 10+ tem no prompt) para realizar requisições de rede, incluindo HTTP. É

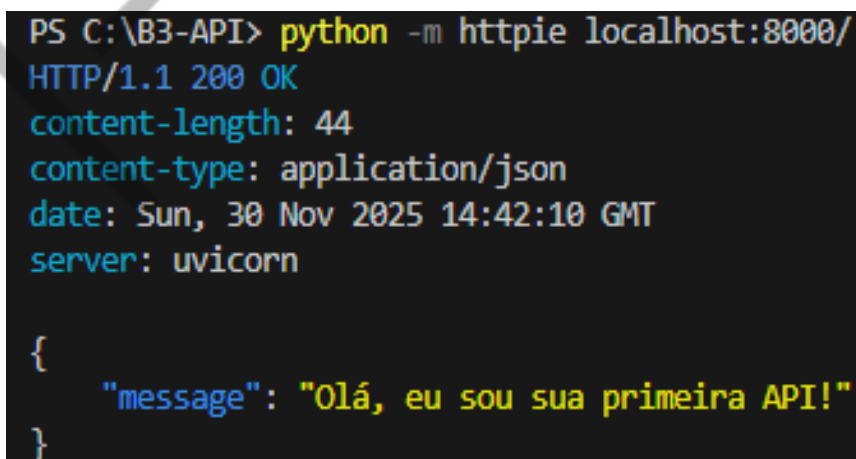
extremamente poderoso e flexível. Por ser CLI, ele é ótimo para scriptar e integrar em pipelines (CI/CD, scripts de deploy, etc.) ou usar em servidores que não têm interface gráfica.

Com o curl você faz qualquer operação HTTP: GET, POST etc. Você especifica o método com -X POST (por padrão sem -X ele faz GET), adiciona cabeçalhos com -H "Header: valor", envia dados com -d '{"json":true}' ou até via arquivo -d @arquivo.json. Suporta autenticação básica facilmente (com -u user:pass) e muitos outros protocolos além de HTTP (FTP etc., mas foquemos em HTTP). No nosso exemplo, já mostramos a chamada curl para testar a API de predição.

Aquela é uma forma bem comum de usar: curl -X POST URL -H "Content-Type:application/json" -d '{...}' . É compacto e replicável. Dá pra colocar isso em um script shell e rodar vários testes sequenciais, por exemplo.

Desvantagens: a sintaxe pode ser intimidadora para iniciantes – cheio de flags, e se a requisição tem muitos cabeçalhos e um payload grande, o comando fica gigantesco e difícil de ler. Porém, há jeitos de contornar: você pode usar -d @arquivo.json para carregar o corpo de um arquivo e até guardar cabeçalhos em arquivos. O importante é que, mesmo com essa verbosidade, o curl é referência e muitas documentações de API fornecem exemplos em curl por ser universal.

Se você dominar alguns parâmetros básicos do curl, consegue testar praticamente qualquer API rapidamente.



```
PS C:\B3-API> python -m httpie localhost:8000/
HTTP/1.1 200 OK
content-length: 44
content-type: application/json
date: Sun, 30 Nov 2025 14:42:10 GMT
server: uvicorn

{
  "message": "Olá, eu sou sua primeira API!"
}
```

Figura 9 – HTTPie
Fonte: Elaborado pelo autor (2026)

Se você gosta da ideia do curl mas quer algo mais user-friendly, conheça o HTTPie. É uma ferramenta de linha de comando alternativa, focada em usabilidade. Uma vez instalada (`pip install httpie` no caso, pois é em Python), ela disponibiliza o comando `http` que tem uma sintaxe mais intuitiva e saída colorida/formatada que facilita ler.

É bem legal para quem vive testando API via terminal, porque poupa tempo e visualmente você entende melhor a resposta. Claro, o HTTPie não tem absolutamente todos os recursos do curl (o curl é monstruosamente completo em suportar protocolos, proxies etc.), mas para 99% dos casos de APIs REST, o HTTPie atende super bem. A desvantagem é só aprender algo novo se você já sabe curl – mas muita gente acha que vale a pena pela simplicidade.

Em resumo, ferramentas de teste de API são suas aliadas no ciclo de desenvolvimento e integração. Use o Postman ou similares (Insomnia é outra GUI popular) quando quiser algo visual, colecionar requisições ou compartilhar com time. Use curl ou HTTPie quando quiser automação, rapidez ou estiver em um ambiente sem interface. O importante é que essas ferramentas agilizam o desenvolvimento e garantem que a API do seu modelo está se comportando como esperado antes mesmo de você codar um cliente no produto final.

Armadilhas comuns (Anti-padrões) em APIs REST

Mesmo conhecendo boas práticas, é fácil escorregar em anti-padrões ao implementar APIs REST. Aqui listamos algumas armadilhas comuns – erros de design ou implementação – e por que devem ser evitadas, especialmente pensando em serviços de ML em produção.

Um dos princípios fundamentais do REST é statelessness, isto é, cada requisição deve ser independente. Um anti-padrão clássico é criar APIs que dependem de sessão no servidor – por exemplo, exigir que o cliente faça um login e daí o servidor guarda alguma sessão em memória e nas próximas requisições daquele cliente assume coisas dessa sessão sem exigir credenciais de novo. Isso quebra a escalabilidade e a robustez: se seu serviço roda em vários servidores (cluster),

sessões de servidor obrigam a ter sticky sessions (o mesmo cliente sempre cair no mesmo nó) ou compartilhar sessão entre nós (o que complica bastante a infra). Além disso, se um servidor cai, as sessões vão junto e os clientes ficam perdidos.

É muito melhor adotar autenticação stateless (como tokens JWT enviados em cada requisição, conforme falamos) e desenhar os endpoints de forma que cada requisição tenha todas as informações necessárias para ser processada isoladamente.

Outra forma de “estado” acidental é fazer o serviço esperar chamadas em ordem: ex., o cliente primeiro chama /init , depois /step1 , etc., e o servidor guarda contexto entre elas. Se por acaso uma chamada falhar ou for perdida, toda aquela sequência se quebra e é difícil recuperar. Se você precisa de uma espécie de fluxo, exponha esse estado como recurso – por exemplo, crie um recurso “/sessao/{id}” que guarda o progresso, mas então cada requisição ainda assim referencia explicitamente esse recurso (mantendo REST, em vez de sessão oculta).

Em resumo: APIs RESTful não devem requerer memória de requisições passadas, cada chamada traz tudo que precisa (no mínimo, o token do usuário ou ID da sessão de negócio etc.). Isso permite que você escale de forma fácil e reinicie serviços sem ninguém perceber.

Já comentamos, mas vale enfatizar como anti-padrão crítico: usar métodos de forma não semântica; com isso você perde todas as vantagens do protocolo e da expressividade e fica impossível para um intermediário ou ferramenta saber o que aquela chamada significa. A solução é seguir as convenções HTTP fielmente. Se uma operação não se encaixa bem num verbo, reavalie o modelo.

Já as URLs verbosas ou “não RESTful” vêm do design de endpoints. Colocar verbos ou detalhes demais na URL, por exemplo:

`/users/getUserById/5` ou `/getAllUsers`

Perceba que isso está duplicando informação (o verbo GET já diz que é pra obter; e o id 5 já indica que é aquele usuário).

`/deleteUser/123` ao invés de DELETE em `/users/123` .

Em geral, se você se pegar escrevendo o verbo na URL ou algo como “get/listAll...”, pare e repense: provavelmente dá pra representar de forma mais REST. Outra armadilha é encher a URL de parâmetros não necessários ou níveis irrelevantes. Por exemplo:

```
/api/v1/usuarios/lista/todos
```

Aqui claramente /api/v1/ usuarios com GET faria a mesma coisa. Ou então misturar maiúsculas, símbolos estranhos etc., ou inconsistências como alguns endpoints no plural, outros no singular, ou misturar português e inglês sem critério.

Tudo isso confunde quem consome, a API deve ter um estilo consistente para nomear caminhos. Algumas dicas: use sempre letras minúsculas e, se precisar separar palavras, hífens (kebab-case) ou underscores (snake_case), mas eleja um e mantenha. Prefira plural para coleções (ex.: /usuarios retornando lista). Seja descritivo(a) mas sucinto nos nomes. Lembre-se: a URL é a identidade do recurso, não precisa conter verbo ou advérbios, somente o “nome” do recurso e hierarquia se fizer sentido.

A exposição excessiva de dados (overexposure) ocorre quando a API retorna muito mais informação do que o necessário, possivelmente incluindo dados sensíveis ou internos. Por exemplo, uma API de previsão que, além do resultado, devolve todo o objeto do modelo ou dados do cliente que não deveriam estar expostos ou uma API de listagem que retorna 10.000 itens quando o cliente só exibirá 20. Isso impacta diretamente a performance e a segurança.

A correção é implementar filtros, paginação e projeção de campos (como discutido em boas práticas), além de sanitizar as respostas para incluir apenas o necessário.

No contexto de ML, imagine um serviço de recomendação que, dado um usuário, retorna itens sugeridos. Se ele também devolver dados pessoais do usuário ou informações de outros usuários, caracteriza-se um caso clássico de overexposure.

Limite sempre a resposta aos dados realmente necessários para a finalidade proposta.

O oposto também é problemático e ocorre quando a API força o cliente a realizar dezenas de chamadas para obter uma informação completa, tornando-se excessivamente “falante” (chatty).

Por exemplo, para montar um resultado completo, o cliente precisa primeiro obter uma lista de IDs e depois, para cada ID, fazer uma nova requisição para os detalhes. Se esse for um padrão de uso comum, é melhor fornecer um endpoint que já retorne as informações completas em uma única chamada, ou que aceite parâmetros para buscar múltiplos registros de uma vez.

Uma API excessivamente granular aumenta a latência por conta do número de round-trips. A solução é ajustar o design dos endpoints ou adotar mecanismos como expand, batch endpoints ou até considerar o uso de GraphQL quando o padrão recorrente for de múltiplas fetches.

Um dos anti-padrões mais graves é expor uma API de Machine Learning sem qualquer autenticação ou proteção sob a justificativa de que “é um servidor interno” ou “ninguém vai descobrir”. Já houve inúmeros casos de vazamentos e uso indevido justamente porque APIs internas, sem autenticação, foram acessadas por terceiros que não deveriam ter permissão. Mesmo em ambientes internos, adote no mínimo um token ou chave de API, qualquer mecanismo que evite acesso casual e garanta rastreabilidade mínima.

Outro erro recorrente é não validar entradas sob a crença de que “é interno” ou “confiável”. Essa é uma armadilha perigosa: mesmo sem um ataque intencional, inputs malformados podem quebrar o serviço, como um JSON gigante pode travar o sistema e strings inesperadas podem gerar exceções.

A validação deve ser rotina: cheque tipos, tamanhos e faixas de valores e sanitize os dados antes de qualquer processamento (por exemplo, escapar strings se forem usadas em consultas SQL, mesmo que hoje o serviço não as insira diretamente).

Em Machine Learning, um exemplo típico: se o modelo espera valores numéricos dentro de um determinado intervalo, verifique isso antes da inferência.

Caso o dado esteja fora do range, retorne um HTTP 422 (Unprocessable Entity) em vez de deixar o modelo gerar um erro obscuro ou um resultado sem sentido.

Em resumo: tratamento de erros e segurança andam de mãos dadas. Não confie na “bondade” do consumidor da API, codifique suposições explícitas e garanta que o sistema falhe de forma previsível, segura e informativa.

EMANIP

MERCADO, CASES E TENDÊNCIAS

Teoria e boas práticas vistas, vamos dar uma olhada em exemplos reais de como APIs REST são utilizadas para servir modelos de Machine Learning em diferentes domínios.

Serviços de predição via Web (MLaaS):

Os grandes provedores de nuvem oferecem Machine Learning as a Service acessível por APIs REST. Por exemplo, a IBM Watson tem vários modelos prontos (de NLP, Visão Computacional, chatbots Watson Assistant) expostos via endpoints REST. Um(a) desenvolvedor(a) pode fazer requisições HTTP aos serviços Watson enviando dados (um texto, uma imagem etc.) e recebe de volta o resultado da análise em JSON – seja uma classificação de sentimento, o reconhecimento de objeto em imagem ou a resposta de um assistente virtual. Isso permite incorporar IA avançada em aplicativos simplesmente consumindo uma API web, sem precisar treinar nada do zero.

Da mesma forma, Google Cloud AI e AWS SageMaker oferecem endpoints REST para modelos hospedados: você envia um POST com os dados e parâmetros e recebe a inferência. Esses casos ilustram como o REST padroniza o acesso a ML independentemente da plataforma ou linguagem – qualquer um que saiba fazer uma requisição web consegue usar modelos sofisticados rodando na nuvem. É o poder da API abstraindo toda complexidade por trás.

Recomendações e personalização:

Empresas como Netflix, Amazon, Spotify são conhecidas pelos sistemas de recomendação. Internamente, esses modelos de recomendação muitas vezes são servidos através de microsserviços com APIs – podendo ser REST ou gRPC.

Por exemplo, o front-end da Netflix, ao carregar a página para um usuário, faz uma chamada a um serviço de recomendação: talvez um GET como

/users/{user_id}/recommendations, que retorna uma lista de filmes/séries recomendados em JSON. Esse serviço, por trás, chama um modelo de ML (um algoritmo colaborativo etc.) para obter os resultados, mas para o front-end ele se apresenta como uma API REST simples. Ao usar REST, o mesmo endpoint pode ser consumido pelos diferentes dispositivos (web, mobile, smart TV) de forma consistente. A Netflix já discutiu publicamente arquiteturas onde serviços RESTful fornecem dados personalizados escalavelmente.

Outro exemplo: o Spotify poderia ter um endpoint do tipo /recommendations?user={id}&limit=20 que retorna um JSON com 20 músicas recomendadas para aquele usuário. O modelo em si pode mudar ao longo do tempo (eles podem trocar o algoritmo por trás, ajustar parâmetros), mas o contrato da API permanece estável – então as apps do Spotify continuam funcionando sem precisar saber o que mudou internamente.

Essa separação via API permite evoluir os modelos de recomendação continuamente sem quebrar a interface com os clientes.

No Brasil, podemos pensar em casos similares: plataformas de e-commerce como Magazine Luiza (Magalu) ou Mercado Livre certamente usam recomendações via APIs para mostrar produtos sugeridos. Quando você abre o app e vê “recomendados para você”, muito possivelmente o front-end fez uma requisição REST a um serviço interno de recomendação com seu userID e recebeu uma lista de produtos. Assim como nos exemplos globais, a API padroniza o acesso ao modelo de ML (no caso, um modelo de recomendação) para diferentes partes do sistema.

Como vemos, APIs REST são a ponte entre o mundo de Machine Learning e as aplicações práticas.

Desde serviços globais até empresas locais, o padrão se repete: encapsular modelos preditivos atrás de endpoints bem definidos. Isso traz todos aqueles benefícios: isolamento, pois o cliente não sabe se por trás você trocou o modelo linear por uma rede neural, contanto que a API continue respondendo no mesmo formato, escalabilidade ao rodar o serviço de ML em várias instâncias e distribuir as requisições, monitoramento unificado com requisições HTTP que deixam rastro claro de uso, latência etc.

Para um(a) engenheiro(a) de ML, pensar a solução já como um serviço com API ajuda a conectar a modelagem ao produto final. Afinal, um modelo só gera impacto se for consumido de maneira confiável pela aplicação e a API é como colocamos esse modelo integrado ao resto do sistema.

EMENDAS

O QUE VOCÊ VIU NESTA AULA?

Nesta aula foi feito um percurso histórico pela Web e pela evolução do protocolo HTTP, mostrando como passamos de uma versão inicial extremamente simples até o moderno HTTP/3 sobre QUIC. Essa trajetória evidenciou como recursos como conexões persistentes e multiplexação melhoraram o desempenho dos serviços web e abriram caminho para APIs mais robustas.

Também foi definido o que é uma API RESTful, seus princípios fundamentais como recursos, verbos HTTP, statelessness e cache e por que o REST se tornou padrão para criar sistemas escaláveis e fáceis de integrar. Além disso, foram comparadas diferentes arquiteturas de API — REST, GraphQL, gRPC, SOAP e RPC — destacando vantagens, limitações e contextos de uso, especialmente em serviços de Machine Learning e Microsserviços.

O conteúdo também detalhou a anatomia de uma requisição e resposta HTTP, analisando método, URL, cabeçalhos e corpo e mostrando como isso se traduz em chamadas reais a modelos de ML via API. Foram apresentadas boas práticas de design de APIs REST, como versionamento, URIs consistentes, uso adequado de códigos de status, paginação, autenticação, caching, documentação e monitoramento. Em contraste, foram discutidos anti-padrões que devem ser evitados, como manter estado no servidor, usar verbos HTTP incorretamente, criar endpoints confusos ou expor dados sensíveis.

Além disso, foram introduzidas ferramentas práticas como Postman, curl e HTTPie, que permitem testar e consumir APIs de forma eficiente, aumentando a produtividade no desenvolvimento.

Essa será a base da nossa disciplina e com esse conhecimento inicial começamos a jornada para transformar modelos em soluções reais, criando APIs bem projetadas que colocam o valor do ML nas mãos de quem precisa.

REFERÊNCIAS

FIELDING (@Fielding28559). **The reason to make a real REST API is to get evolvability...** ... 2013. Disponível em: <https://x.com/fielding/status/376835835670167552>. Acesso em: 14 jan. 2026.

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps: Operationalizing Machine Learning Models**. Sebastopol: O'Reilly Media, 2021. KORIYAMA, A. **RESTの力 / The Power of REST**. 2019. Disponível em: <https://speakerdeck.com/koriym/the-power-of-rest-24bf1cf0-1af8-45dd-bce6-6a5553511775>. Acesso em: 14 jan. 2026.

LUBANOVIC, B. **FastAPI: Modern Python Web Development**. Sebastopol: O'Reilly Media, 2023.

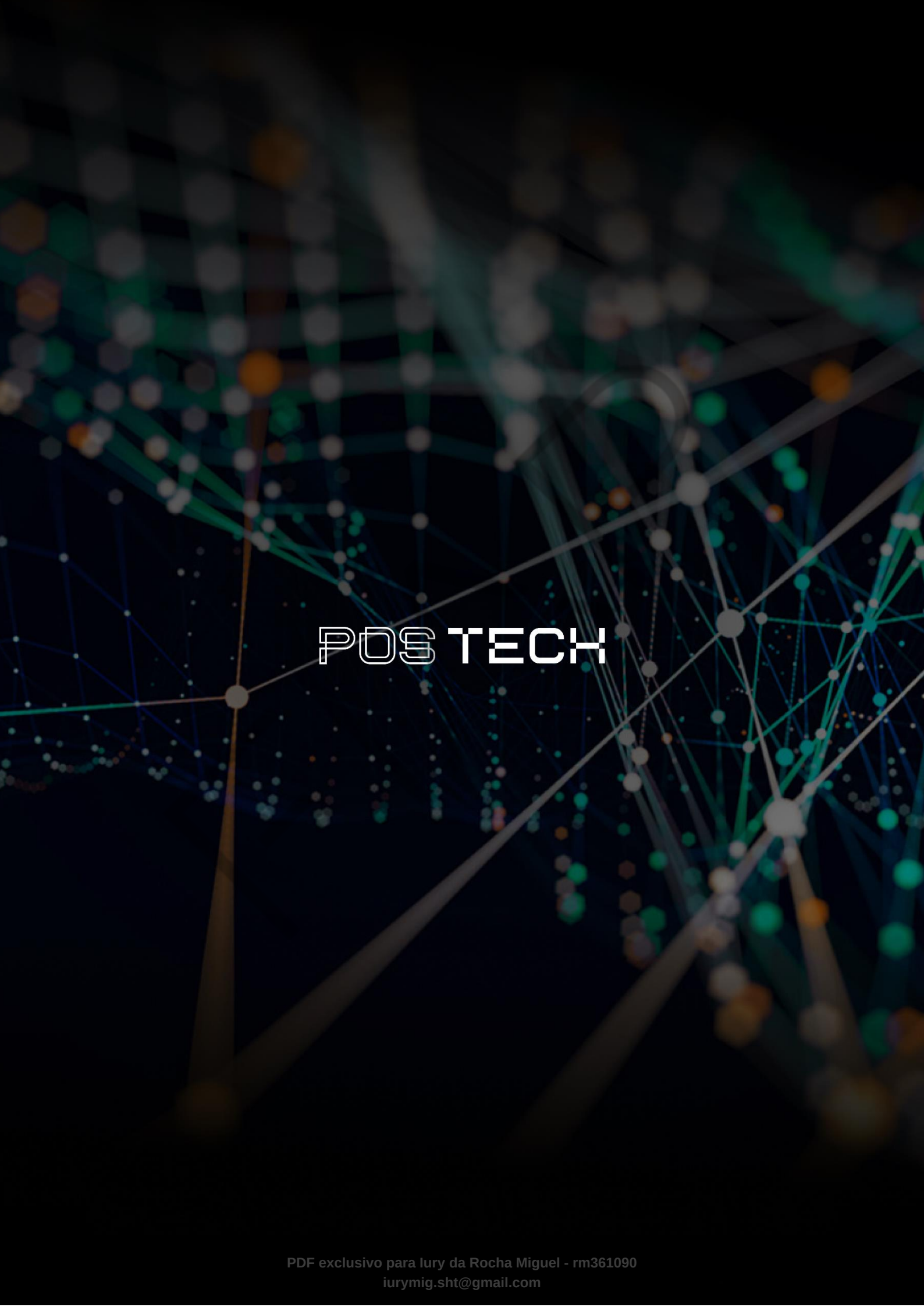
NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2021.

TOLEDO, V. **Quem é Tim Berners-Lee? Conheça o criador da World Wide Web (WWW)**. 2026. Disponível em: <https://tecnoblog.net/responde/quem-e-tim-berners-lee-conheca-o-criador-da-world-wide-web-www/>. Acesso em: 14 jan. 2026.

PALAVRAS-CHAVE

API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON. MLOps.

EMEND



POSTECH